

CashlessIQ — Build Plan & Technical Spec

Sep 29, 2026 · @Create a new relic monitor to track successful login process. Th

Purpose and scope

This is the build contract for **CashlessIQ**: a Snowflake-native copilot that turns a hospital's cashless pre-authorisation request into a cited, reviewable decision within IRDAI's one-hour window. It is written for Claude Code (and any teammate) to implement from; the companion *Research & Strategy* doc explains why. Deadline: prototype submitted by **4 Oct 2026, 6 PM IST** (portal closes 11:59 PM IST).

MVP scope

Must have (the submission fails without it)	Stretch (only after every must is green)	Out of scope
Idempotent setup script for all Snowflake objects	SLA Task + Alert to Slack or email at 45 minutes	Real patient or insurer data of any kind
Synthetic members, policies, claims + 30 pre-auth PDFs with golden outcomes	Live second-policy onboarding via the CoCo skill	Auto-denial by AI (a human always decides a denial)
PDF → structured facts via AI_PARSE_DOCUMENT + AI_EXTRACT	Row access policy by TPA	Final discharge authorisation (only the initial pre-auth)
Policy clauses indexed in Cortex Search with stable clause IDs	Java UDF (e.g. ICD-10 format validation)	Hospital-facing portal, payments, emails to patients
MEMBER_360 semantic view with verified queries	Snowflake Marketplace join (US mode)	Multi-language documents
Deterministic rules tools (waiting periods, room-rent proportion, sub-limits, SLA) with unit tests	CoWork (Snowflake Intelligence) front end for the agent	Mobile UI
Cortex Agent producing a schema-valid JSON decision with a citation on every line		
Streamlit (container runtime) reviewer console + decision log		

Must have (the submission fails without it)	Stretch (only after every must is green)	Out of scope
Masking policy on clinical text and member PII, 3 role-based demo users		
Golden-set evaluation with an accuracy scorecard		
<code>policy-onboarding</code> CoCo skill in <code>.cortex/skills/</code>		
README, dataset licenses, CoCo prompt playbook, deck, judge user		

Who does what: Claude Code vs CoCo CLI

The rules make **CoCo CLI use mandatory** and judged, so its use must be real and visible — not a token call.

Work	Tool	Evidence kept in repo
Repo scaffolding, Python packages, generators, unit tests, Streamlit code, README	Claude Code	Git history
Pure-Python rules engine (no Snowflake dependency) + tests	Claude Code	<code>tests/</code> passing
Creating and tuning the semantic view, search service and agent; debugging SQL, grants and runtime errors inside Snowflake	CoCo CLI (<code>agent-studio</code> , <code>search-optimization</code> , <code>document-intelligence</code> , <code>developing-with-streamlit</code> skills)	Prompts + outcomes in <code>docs/coco-playbook.md</code> ; generated SQL committed under <code>sql/</code>
Custom <code>policy-onboarding</code> skill	Written with CoCo's <code>skill-development</code> skill	<code>.cortex/skills/policy-onboarding/SKILL.md</code>
Cost check and governance audit	CoCo (<code>cost-intelligence</code> , <code>data-governance</code>)	Screenshots/notes in playbook

Rule of thumb: **Claude Code owns the files; CoCo owns anything that talks to Snowflake interactively.** Anything CoCo creates is exported back to `sql/` so the setup script can rebuild the whole account from scratch. Optionally, the Snowflake AI Kit ships a Claude Code plugin that routes Snowflake-intent requests from Claude Code to CoCo CLI ([snowflake-ai-kit](#)) — useful, but log those CoCo runs in the playbook too.

Engineering principles

These keep the foundation solid so later days add features instead of fixing plumbing.

1. **Domain core is pure Python.** All money and date logic lives in `src/cashlessiq/rules/` with zero Snowflake imports, fully unit-tested locally. Snowpark stored procedures are thin wrappers that load rows, call the core, and return JSON.
2. **The LLM never computes numbers.** Every rupee amount and every waiting-period verdict comes from a tool. The agent selects tools, explains results and attaches citations.
3. **Every claim is traceable.** Each decision line carries at least one citation: a `clause_id` (policy text) or a record reference (`table:primary_key`). A line without a citation fails schema validation.
4. **Rules are data, not code.** Sub-limits, caps, co-pay and waiting periods live in `DOCS.POLICY_RULE` rows linked to a `clause_id`, never hard-coded. Adding a policy product = adding rows.
5. **Infrastructure as code, idempotent.** Every Snowflake object is created by numbered SQL files using `CREATE OR REPLACE / IF NOT EXISTS`. `make setup` rebuilds the account from nothing; `make teardown` removes it.
6. **Humans decide denials.** The agent may output APPROVE, APPROVE_WITH_DEDUCTIONS, QUERY or REFER — never DENY.
7. **Synthetic only, reproducible.** All generators take a fixed seed; re-running produces identical data and identical PDFs.
8. **Least privilege.** The app and agent run under roles with only the grants they need; ACCOUNTADMIN is used only in `00_account.sql`.
9. **Cost-aware by default.** XS warehouse, 60-second auto-suspend, cheapest adequate model per task, resource monitor from day 0.

Conventions

Area	Convention
Language	Python 3.11 (matches Snowflake container runtime); type hints; <code>ruff + pytest</code>

Area	Convention
Snowflake CLI	<code>snow</code> with connection name <code>cashlessiq</code> in <code>~/.snowflake/connections.toml</code> ; never commit credentials
Object names	UPPER_SNAKE_CASE; tables singular nouns (<code>MEMBER</code> , <code>POLICY</code>); tools prefixed <code>T00L_</code> ; views prefixed <code>V_</code>
IDs	Readable string keys: <code>MEM-000123</code> , <code>POL-000123</code> , <code>PAR-0007</code> (pre-auth request), <code>DEC-...</code> ; clause IDs <code><PRODUCT>-<VERSION>-<SECTION></code> e.g. <code>ASP-RGI-V1-4.3</code>
Money	Integer rupees (<code>NUMBER(12,0)</code>), column suffix <code>_INR</code> ; no floats for money
Time	<code>TIMESTAMP_TZ</code> in IST for request/decision times; dates as <code>DATE</code>
SQL files	Numbered, one concern per file, each re-runnable
Secrets	<code>.env</code> (git-ignored) + <code>.env.example</code> ; Streamlit uses the session, not passwords
Commits	Small, conventional (<code>feat:</code> , <code>fix:</code> , <code>chore:</code>); each phase ends with a tag <code>phase-N</code>

Repository structure

Repo name: `cashlessiq` (public on GitHub at submission; MIT license).

```

cashlessiq/
├── README.md                # problem, architecture, setup, demo script,
├── datasets + licenses
├── LICENSE
├── Makefile                 # setup, data, deploy, test, eval, teardown
├── pyproject.toml           # package + dev deps (ruff, pytest, reportlab,
faker)
├── snowflake.yml            # snow CLI project: Streamlit app + Snowpark
procs
├── .env.example
├── .cortex/
│   ├── skills/
│   │   └── policy-onboarding/
│   │       └── SKILL.md      # custom CoCo skill

```

```

└─ sql/
  │ └─ 00_account.sql          # ACCOUNTADMIN only: roles, warehouse,
resource monitor, cross-region
  │ └─ 01_database.sql        # database, schemas, stages, file formats
  │ └─ 02_core_tables.sql
  │ └─ 03_docs_tables.sql
  │ └─ 04_app_eval_tables.sql
  │ └─ 10_load_data.sql        # COPY INTO from stages
  │ └─ 20_extract_preauth.sql  # AI_PARSE_DOCUMENT + AI_EXTRACT →
PREAUTH_FACTS
  │ └─ 21_policy_clauses.sql   # parse + chunk wordings → POLICY_CLAUSE
  │ └─ 22_search_service.sql   # CLAUSE_SEARCH
  │ └─ 30_semantic_view.sql     # MEMBER_360 (exported from CoCo agent-studio)
  │ └─ 40_tools.sql            # TOOL_* procedure registration (if not via
snowflake.yml)
  │ └─ 50_agent.sql            # CASHLESSIQ_AGENT (exported)
  │ └─ 60_governance.sql       # masking + row access policies, demo users
  │ └─ 70_alerts.sql           # stretch: SLA task + alert
  └─ 99_teardown.sql
└─ src/cashlessiq/
  │ └─ rules/                  # PURE PYTHON domain core
  │   └─ models.py            # dataclasses: Policy, Rule, PreAuthFacts,
DecisionLine...
  │   └─ waiting_periods.py
  │   └─ payable.py           # room-rent proportion, sub-limits, co-pay, SI
cap
  │   └─ sla.py
  │   └─ procs/                # Snowpark wrappers → TOOL_* procedures
  │   └─ decision_schema.py    # JSON schema + validator for agent output
  │   └─ agent_client.py       # calls agent:run, parses and validates output
└─ data_gen/
  │ └─ synthea_to_core.py      # Synthea CSV → MEMBER/POLICY/CONDITION/CLAIM
CSVs
  │ └─ policies.py            # sum insured, inception dates, TPA assignment
  │ └─ preauth_pdfs.py        # 30 request PDFs from case specs
  │ └─ golden_cases.yaml      # case specs = single source of truth for PDFs
+ expected outcomes
  │   └─ output/              # generated, git-ignored except small samples
└─ agent/
  │ └─ instructions.md        # orchestration + response instructions
  │ └─ tools.yaml             # tool names, descriptions, input schemas
└─ app/
  │ └─ streamlit_app.py       # entry; pages below
  │ └─ pages/                 # 1_Queue, 2_Case, 3_Decision_Log, 4_Scorecard
  │   └─ environment.yml / requirements.txt
└─ eval/

```

```

|   └─ run_eval.py           # runs golden set through agent, writes EVAL
tables
└─ tests/                   # pytest for rules core + schema validator
   └─ docs/
      ├── architecture.md
      ├── coco-playbook.md   # every CoCo prompt used + what it produced
      ├── datasets.md        # sources + licenses (rules §4.3b)
      ├── demo-script.md
      └─ deck/               # presentation

```

Snowflake account layout

One database, `CASHLESSIQ`, split by concern so grants stay simple.

Schema	Holds	Written by
<code>RAW</code>	Synthea CSVs as loaded, untouched	Load scripts
<code>CORE</code>	Members, policies, conditions, claims, hospitals, TPAs	Load scripts
<code>DOCS</code>	Stages for PDFs; pre-auth requests and extracted facts; policy clauses and rules	Extraction pipeline, policy onboarding
<code>AI</code>	Cortex Search service, semantic view, agent, <code>TOOL_*</code> procedures	CoCo / deploy
<code>APP</code>	Decisions, review actions, SLA events, Streamlit app	App + agent client
<code>GOV</code>	Masking and row access policies, user→TPA map	Governance script
<code>EVAL</code>	Golden cases, eval runs and results	Eval runner

Compute and cost controls

Object	Setting
Warehouse <code>CIQ_WH</code>	<code>XSMALL</code> , <code>AUTO_SUSPEND = 60</code> , <code>AUTO_RESUME = TRUE</code> , <code>INITIALLY_SUSPENDED = TRUE</code>
Resource monitor <code>CIQ_RM</code>	Notify at 50% and 75%, suspend at 90% of the credit quota you set

Object	Setting
Compute pool for Streamlit container runtime	Smallest CPU pool; auto-suspend; stop the app when not demoing
Cross-region inference	<code>CORTEX_ENABLED_CROSS_REGION</code> checked in <code>00_account.sql</code> ; set to allow if disabled (agents need it)

Roles and demo users

Cortex Agents use the user's **default role**, not the session's active role, so each persona is a separate user with its own default role.

Role	Purpose	Sees clinical notes	Sees member name/phone	Can approve
<code>CIQ_ADMIN</code>	Owns all objects; used by setup and CoCo	Yes	Yes	—
<code>CIQ_MEDICAL_OFFICER</code>	Reviews and finalises decisions	Yes	Yes	Yes
<code>CIQ_PROCESSOR</code>	Handles queue, raises queries	Masked	Partially masked	No (can only draft/query)
<code>CIQ_AUDITOR</code>	Read-only audit of decisions and logs	Masked	Masked	No

Users: `CIQ_MO_USER` , `CIQ_PROC_USER` , `CIQ_AUDIT_USER` (one per role above), plus `CIQ_JUDGE` (default role `CIQ_MEDICAL_OFFICER`) for evaluators. All personas inherit a shared `CIQ_APP_USER` role that grants `SNOWFLAKE.CORTEX_AGENT_USER` , usage on the agent, search service, semantic view, procedures, warehouse and Streamlit app.

Data model

A pre-auth request links a policy (and through it a member and product) to extracted facts; a decision links back to the request and cites clauses and records. Columns below are the minimum; add more only if a tool or the app needs them.

erDiagram

```

TPA ||--o{ POLICY : administers
POLICY_PRODUCT ||--o{ POLICY : "sold as"
MEMBER ||--o{ POLICY : holds
MEMBER ||--o{ MEMBER_CONDITION : has
POLICY ||--o{ CLAIM : "past claims"
HOSPITAL ||--o{ PREAUTH_REQUEST : sends
POLICY ||--o{ PREAUTH_REQUEST : "requested under"
PREAUTH_REQUEST ||--|| PREAUTH_FACTS : "extracted to"
POLICY_PRODUCT ||--o{ POLICY_CLAUSE : contains
POLICY_CLAUSE ||--o{ POLICY_RULE : "source of"
PREAUTH_REQUEST ||--o{ DECISION : "decided by"
DECISION ||--o{ REVIEW_ACTION : "reviewed via"
PREAUTH_REQUEST ||--o{ GOLDEN_CASE : "expected outcome"

```

CORE

Table	Key columns
TPA	TPA_ID , NAME
HOSPITAL	HOSPITAL_ID , NAME , CITY , IS_NETWORK
POLICY_PRODUCT	PRODUCT_ID , PRODUCT_NAME , INSURER , UIN , VERSION , EFFECTIVE_FROM , SOURCE_URL
MEMBER	MEMBER_ID , SYNTHESA_ID , FULL_NAME , DOB , GENDER , CITY , PHONE
POLICY	POLICY_ID , MEMBER_ID , PRODUCT_ID , TPA_ID , SUM_INSURED_INR , INCEPTION_DATE , CONTINUOUS_COVER_SINCE , CUMULATIVE_BONUS_INR , STATUS
MEMBER_CONDITION	MEMBER_ID , ICD10_CODE , CONDITION_NAME , FIRST_DIAGNOSED_ON , DECLARED_AT_PROPOSAL
CLAIM	CLAIM_ID , POLICY_ID , ADMISSION_DATE , DISCHARGE_DATE , ICD10_CODE , CLAIMED_INR , PAID_INR , STATUS

DOCS

Object	Key columns / notes
Stage <code>PREAUTH_STAGE</code>	Request PDFs; directory table on; server-side encryption (required by <code>AI_PARSE_DOCUMENT</code>)
Stage <code>POLICY_STAGE</code>	Policy wording PDFs
<code>PREAUTH_REQUEST</code>	<code>REQUEST_ID</code> , <code>FILE_PATH</code> , <code>POLICY_ID</code> , <code>HOSPITAL_ID</code> , <code>RECEIVED_AT</code> , <code>STATUS</code> (NEW / DRAFTED / QUERIED / DECIDED)
<code>PREAUTH_FACTS</code>	<code>REQUEST_ID</code> , <code>DIAGNOSIS</code> , <code>ICD10_CODE</code> , <code>PROCEDURE</code> , <code>IS_EMERGENCY</code> , <code>ADMISSION_DATE</code> , <code>PLANNED_LOS_DAYS</code> , <code>ROOM_CATEGORY</code> , <code>ROOM_RENT_PER_DAY_INR</code> , <code>ICU_DAYS</code> , <code>COST_BREAKUP</code> (VARIANT: room, ICU, surgeon, OT, medicines, consumables, implants, diagnostics), <code>ESTIMATED_TOTAL_INR</code> , <code>CLINICAL_NOTE</code> , <code>MISSING_FIELDS</code> (ARRAY), <code>RAW_EXTRACT</code> (VARIANT)
<code>POLICY_CLAUSE</code>	<code>CLAUSE_ID</code> , <code>PRODUCT_ID</code> , <code>SECTION</code> , <code>HEADING</code> , <code>TEXT</code> , <code>PAGE</code>
<code>POLICY_RULE</code>	<code>RULE_ID</code> , <code>PRODUCT_ID</code> , <code>RULE_TYPE</code> , <code>PARAMS</code> (VARIANT), <code>CLAUSE_ID</code> , <code>VERIFIED_BY</code> , <code>VERIFIED_AT</code>

`RULE_TYPE` values: `ROOM_RENT_CAP`, `ICU_CAP`, `PROCEDURE_SUBLIMIT`, `COPAY`, `WAITING_INITIAL`, `WAITING_SPECIFIC`, `WAITING_PED`, `EXCLUSION`, `PROPORTIONATE_DEDUCTION`. Example `PARAMS` for a room cap: `{"pct_of_si": 2, "max_inr_per_day": 5000}`.

APP and EVAL

Table	Key columns
<code>APP.DECISION</code>	<code>DECISION_ID</code> , <code>REQUEST_ID</code> , <code>AGENT_RUN_ID</code> , <code>MODEL</code> , <code>OUTCOME</code> , <code>PAYABLE_INR</code> , <code>DECISION_JSON</code> (VARIANT), <code>SCHEMA_VALID</code> , <code>LATENCY_MS</code> , <code>CREATED_AT</code>

Table	Key columns
APP.REVIEW_ACTION	ACTION_ID , DECISION_ID , REVIEWER , ROLE , ACTION (ACCEPT / EDIT / QUERY / REFER), FINAL_PAYABLE_INR , COMMENT , ACTED_AT
APP.SLA_EVENT	REQUEST_ID , EVENT (RECEIVED / DRAFTED / DECIDED / BREACH_WARNING), AT
EVAL.GOLDEN_CASE	REQUEST_ID , EXPECTED_OUTCOME , EXPECTED_PAYABLE_INR , TOLERANCE_INR , EXPECTED_CLAUSE_IDS (ARRAY), TAGS (ARRAY)
EVAL.EVAL_RUN / EVAL.EVAL_RESULT	Run metadata (model, time, git SHA); per-case outcome match, amount match, citation recall, schema validity, latency

Synthetic data and golden cases

Decision: use our own seeded Python generator, not Synthea, for the MVP. Synthea is a Java toolchain producing US names, cities and SNOMED codes; remapping it costs a day we don't have. A Faker `en_IN` generator with a curated ICD-10 condition list gives Indian-realistic data in an hour. Synthea import stays a stretch.

Volumes (seed = 42)

Entity	Count	Notes
TPA	3	Fictional names
Hospital	25	Pune, Mumbai, Bengaluru, Hyderabad, Delhi; ~80% network
Member / Policy	500 / 500	Sum insured ₹3L–₹10L in ₹50k steps; inception dates spread over the last 6 years
Member condition	~800	From a curated list of ~25 conditions with ICD-10 codes (diabetes, hypertension, cataract, hernia, knee osteoarthritis, gallstones, etc.); some declared at proposal (PED)
Past claim	~300	Used for the 360 view and sum-insured consumed

Entity	Count	Notes
Pre-auth request PDFs	30 golden + 20 unlabelled	Unlabelled ones make the queue look real

Policy rules source

Pick **one** published Arogya Sanjeevani wording as the primary product (IRDAI hosts insurers' wordings — e.g. [this sample](#)) and record its UIN and version in `POLICY_PRODUCT`. Rule values in `POLICY_RULE` are extracted from that PDF and **verified by a human** before any golden case is written — wordings differ by insurer and version, so never mix values from different PDFs.

Golden case matrix (30 cases)

Tag	Cases	Expected outcome	What it tests
<code>clean</code>	6	APPROVE	Happy path, citations for coverage clause
<code>room_over_cap</code>	5	APPROVE_WITH_DEDUCTIONS	Room-rent cap + proportionate deduction clause
<code>sublimit</code>	4	APPROVE_WITH_DEDUCTIONS	Procedure sub-limit (e.g. cataract per eye)
<code>waiting_initial</code>	3	REFER	Admission within the initial waiting period, non-accident
<code>accident_exempt</code>	2	APPROVE	Initial waiting period does not apply to accidents
<code>waiting_specific</code>	3	REFER	Listed condition inside its specific waiting period
<code>waiting_ped</code>	3	REFER	Declared pre-existing condition inside the PED window
<code>missing_info</code>	2	QUERY	Missing cost break-up or diagnosis; extraction flags it

Tag	Cases	Expected outcome	What it tests
si_exhaustion	1	APPROVE_WITH_DEDUCTIONS	Remaining sum insured lower than estimate
exclusion	1	REFER	Treatment in the policy's exclusion list

`data_gen/golden_cases.yaml` is the single source of truth: each case spec drives both the PDF generator and the `EVAL.GOLDEN_CASE` row. **Expected amounts are hand-calculated** and the same numbers are asserted in `tests/` for the rules core — otherwise the eval would just grade the tools against themselves.

Pre-auth PDF layout

Two pages, generic layout (do not copy any real insurer's or TPA's form): page 1 = hospital, patient, policy number, admission date, diagnosis, procedure, room category, planned stay, cost break-up table; page 2 = a short clinical note in free text. Vary wording and ordering across PDFs so extraction is genuinely tested.

Component specs

Exact Snowflake SQL syntax for AI functions, search services, semantic views and agents changes often — **confirm each statement with the matching CoCo skill before committing it**, then save the working version under `sql/`.

1. Pre-auth document pipeline (`20_extract_preauth.sql`)

- Input: PDFs on `DOCS.PREAUTH_STAGE` ; one `PREAUTH_REQUEST` row per file with `RECEIVED_AT` .
- Step 1: `AI_PARSE_DOCUMENT` in layout mode → text/markdown per file (kept for display and fallback).
- Step 2: `AI_EXTRACT` with a fixed response format for every `PREAUTH_FACTS` field → typed columns + `RAW_EXTRACT` .
- Validation: required fields = diagnosis, procedure, admission date, room category, room rent/day, estimated total, cost break-up. Any null → appended to `MISSING_FIELDS` .
- Acceptance: ≥ 90% field-level accuracy on the 30 golden PDFs vs `golden_cases.yaml` .
- Stretch: convert to a Dynamic Table so new PDFs process automatically (`ai-functions-pipeline-builder` skill).

2. Policy clauses and search (21_policy_clauses.sql , 22_search_service.sql)

- Parse the wording PDF in layout mode; split on numbered section headings; one row per clause in `POLICY_CLAUSE` with a stable `CLAUSE_ID`.
- Cortex Search service `AI.CLAUSE_SEARCH` on `TEXT`, attributes `PRODUCT_ID`, `SECTION`, `HEADING`, warehouse `CIQ_WH`, long target lag (the corpus rarely changes).
- The agent always filters by the request's `PRODUCT_ID`.
- Acceptance: 10 probe questions (e.g. "room rent limit", "cataract", "pre-existing disease waiting period") return the correct clause in the top 3.

3. Semantic view `AI.MEMBER_360` (30_semantic_view.sql)

- Tables: `MEMBER`, `POLICY`, `POLICY_PRODUCT`, `TPA`, `MEMBER_CONDITION`, `CLAIM`, `PREAUTH_REQUEST`, `PREAUTH_FACTS`, with relationships on the keys in the data model.
- Metrics: `POLICY_TENURE_MONTHS`, `SI_CONSUMED_INR` (paid claims in current policy year), `SI_REMAINING_INR`, `CLAIMS_LAST_3Y`, `DECLARED_PED_COUNT`.
- Synonyms: "sum insured"/"SI"/"cover"; "PED"/"pre-existing"; "pre-auth"/"cashless request".
- 10 verified queries, e.g. "remaining sum insured for policy POL-000123", "conditions declared at proposal for member MEM-000123", "claims in the last 3 years for this member".
- Build with CoCo `agent-studio`; export DDL to the SQL file.

4. Rules tools (src/cashlessiq/rules/ + procs/)

Pure-Python core, wrapped as Snowpark stored procedures in schema `AI`. Tools return **IDs, numbers and clause IDs only — no member PII or clinical text**, so masking stays enforced at the table and app layer (owner's-rights procedures would otherwise bypass it).

Tool	Input	Output (JSON)
<code>TOOL_GET_CASE</code>	<code>request_id</code>	Facts (non-PII), <code>policy_id</code> , <code>product_id</code> , sum insured, SI remaining, inception and continuous-cover dates, declared ICD-10 codes, missing fields
<code>TOOL_CHECK_WAITING_PERIODS</code>	<code>request_id</code>	List of { <code>rule_id</code> , <code>clause_id</code> , <code>rule_type</code> , <code>applies</code> , <code>months_elapsed</code> , <code>months_required</code> , <code>reason</code> }

Tool	Input	Output (JSON)
<code>TOOL_COMPUTE_PAYABLE</code>	<code>request_id</code>	<code>{lines: [{item, claimed_inr, payable_inr, deduction_inr, rule_id, clause_id, calc}], total_claimed_inr, total_payable_inr, si_remaining_inr}</code>
<code>TOOL_SLA_STATUS</code>	<code>request_id</code>	<code>{received_at, elapsed_min, remaining_min, state: GREEN / AMBER / RED}</code> (amber from 30 min, red from 45)

`TOOL_COMPUTE_PAYABLE` order of operations: (1) room/ICU caps from `ROOM_RENT_CAP` / `ICU_CAP`; (2) if actual room rent exceeds the cap, apply the ratio *eligible ÷ actual* to exactly the expense categories listed in the `PROPORTIONATE_DEDUCTION` rule's `PARAMS.applies_to` (taken from the wording — never assumed); (3) procedure sub-limits; (4) co-pay if the product has one; (5) cap at remaining sum insured. Every step that changes an amount emits a line with its `rule_id`, `clause_id` and a human-readable `calc` string (e.g. `"50000/80000 × 600000 = 375000"`).

5. Agent `AI.CASHLESSIQ_AGENT` (`agent/`, `50_agent.sql`)

- Tools: `MEMBER_360` (Analyst), `CLAUSE_SEARCH` (Search), the four `TOOL_*` procedures.
- Orchestration instructions (in `agent/instructions.md`):
 - Call `TOOL_GET_CASE` first. If `missing_fields` is non-empty → outcome `QUERY` listing what the hospital must send.
 - Call `TOOL_CHECK_WAITING_PERIODS`. Any `applies = true` or an exclusion match → outcome `REFER`, citing the clause.
 - Otherwise call `TOOL_COMPUTE_PAYABLE`; outcome `APPROVE` if no deductions, else `APPROVE_WITH_DEDUCTIONS`.
 - For every clause cited, retrieve its text via `CLAUSE_SEARCH` filtered to the product, so the reviewer sees the words.
 - Use `MEMBER_360` only for context questions (history, tenure) — never to recompute amounts.
 - Never perform arithmetic; never output `DENY`; respond with JSON only, matching the decision schema.
- Client (`agent_client.py`): call the agent run API, extract the JSON, validate against the schema; on failure retry once with the validation error appended; persist to `APP.DECISION` with `SCHEMA_VALID`.

6. Decision schema (`decision_schema.py`)

```
{
  "request_id": "PAR-0007",
  "outcome": "APPROVE_WITH_DEDUCTIONS",
  "total_claimed_inr": 185000,
  "total_payable_inr": 152300,
  "lines": [
    {
      "item": "Room rent (4 days)",
      "claimed_inr": 32000,
      "payable_inr": 20000,
      "deduction_inr": 12000,
      "reason": "Room rent above cap of 2% of sum insured, max per day",
      "calc": "min(2% × 300000, 5000) × 4 = 20000",
      "citations": [
        {"type": "clause", "id": "ASP-XXX-V1-4.1"},
        {"type": "record", "id": "CORE.POLICY:POL-000123"}
      ]
    }
  ],
  "queries": [],
  "refer_reasons": [],
  "summary": "Two-sentence plain-English explanation for the reviewer.",
  "sla": {"elapsed_min": 12, "state": "GREEN"}
}
```

Validation rules: `outcome` ∈ {APPROVE, APPROVE_WITH_DEDUCTIONS, QUERY, REFER}; every line has ≥ 1 citation; `total_payable_inr` equals the sum of line `payable_inr`; QUERY requires non-empty `queries`; REFER requires non-empty `refer_reasons`. (Numbers above are illustrative only.)

7. Streamlit reviewer console (`app/`)

Container runtime is mandatory — Cortex Agent APIs are not supported on the warehouse runtime.

Page	Shows	Actions
Queue	Requests with hospital, member (masked per role), estimate, SLA timer coloured green / amber / red	Open a case

Page	Shows	Actions
Case	Extracted facts, PDF preview, member 360 summary, decision card: outcome, line items, <code>calc</code> , citation chips that expand clause text or the record	Run CashlessIQ; Accept / Edit amount / Raise query / Refer (buttons enabled by role)
Decision log	All decisions and review actions with timings	Filter by outcome, reviewer, SLA state
Scorecard	Latest eval run: outcome accuracy, amount accuracy, citation recall, schema validity, p50 latency	Compare runs

The header always shows the signed-in user and role, so the masking demo is obvious.

8. Governance (`60_governance.sql`)

- Masking `GOV.MASK_CLINICAL_TEXT` on `PREAUTH_FACTS.CLINICAL_NOTE` : clear for `CIQ_MEDICAL_OFFICER` , `***` otherwise.
- Masking `GOV.MASK_PII` on `MEMBER.FULL_NAME` , `MEMBER.PHONE` : clear for medical officer; initials / last-4 for processor; fully masked for auditor.
- Use `IS_ROLE_IN_SESSION` in policy bodies so role inheritance works.
- Stretch: row access policy on `POLICY` and `PREAUTH_REQUEST` via `GOV.USER_TPA_MAP` .
- Acceptance: the same query run as each demo user returns the three expected views of the data.

9. Evaluation (`eval/run_eval.py`)

Metric	Definition	Target
Outcome accuracy	Agent outcome = expected outcome	≥ 90%
Amount accuracy	payable – expected ≤ tolerance (APPROVE cases)	≥ 90%
Citation recall	Expected clause IDs ⊆ cited clause IDs	≥ 80%
Schema validity	Output passes validator on first or retry attempt	100%
Latency p50	Request to validated decision	Report it

Results go to `EVAL.EVAL_RESULT` tagged with model and git SHA; the Scorecard page reads them.

10. CoCo skill `policy-onboarding`

`SKILL.md` steps: upload a new wording PDF to `POLICY_STAGE` → parse and chunk into `POLICY_CLAUSE` → propose `POLICY_RULE` rows **as a diff for human approval** (never auto-insert) → on approval insert rules and refresh `CLAUSE_SEARCH` → generate 3 test requests for the new product and run them through the agent. Written with CoCo's `skill-development` skill; its run transcript goes in the playbook.

Build phases and exit criteria

Each phase ends with a git tag and a green exit check; don't start the next phase until the current one passes. The one allowed overlap: the pure-Python rules core (Phase 3a) can be built any time after Phase 0, since it needs no Snowflake.

Phase	When	Deliverables	Exit criteria (definition of done)
0 — Foundations	Tue night → Wed noon	Repo scaffold per structure above; <code>pyproject.toml</code> , <code>Makefile</code> , <code>ruff</code> + <code>pytest</code> ; <code>.env.example</code> ; <code>sql/00-04</code> ; snow connection <code>cashlessiq</code> ; resource monitor; spike : a Streamlit container-runtime app that calls a trivial Cortex Agent and prints the reply	<code>make setup</code> runs twice with no errors; <code>make test</code> green; <code>cortex</code> connects and lists <code>CASHLESSIQ</code> schemas; <code>spike</code> returns an agent reply (or a documented fallback is chosen)
1 — Data	Wed	<code>data_gen/</code> generators; <code>golden_cases.yaml</code> (30 cases, hand-calculated amounts); 50 PDFs; <code>sql/10_load_data.sql</code> ; primary policy wording chosen and rules verified	Row counts match the volumes table; every golden case has a PDF and an expected outcome; <code>POLICY_RULE</code> rows reviewed by a human
2 — Documents and retrieval	Thu	<code>sql/20-22</code> , <code>sql/30</code> ; <code>MEMBER_360</code> with 10 verified queries	Extraction ≥ 90% field accuracy on golden PDFs; search probes top-3 correct 10/10; verified queries all pass

Phase	When	Deliverables	Exit criteria (definition of done)
3a — Rules core	Wed-Fri	<code>src/cashlessiq/rules/*</code> + tests covering every golden tag	100% of rules tests pass; each golden amount reproduced by hand-calculated test
3b — Tools and agent	Fri	<code>TOOL_*</code> procedures; <code>agent/instructions.md</code> , <code>tools.yaml</code> ; <code>sql/50_agent.sql</code> ; <code>agent_client.py</code> + schema validator	10 golden cases through the agent: schema-valid 10/10, outcome correct $\geq$ 8/10
4 — App and governance	Sat	Streamlit pages; <code>sql/60_governance.sql</code> ; demo users; <code>policy-onboarding</code> skill; (stretch) <code>sql/70_alerts.sql</code>	Full demo flow works as each persona; masking differs correctly across the three users; skill runs once end to end
5 — Prove and ship	Sun, submit by 6 PM IST	Eval run on all 30; Scorecard page; README, <code>docs/*</code> , dataset licenses, CoCo playbook; judge user; 10-slide deck	Eval meets targets (or gaps documented honestly); fresh-clone <code>make setup</code> && <code>make data</code> && <code>make deploy</code> works; judge login tested from a private window; submission form complete

Daily rhythm

- Start of day: re-read this phase's exit criteria; open a CoCo session for Snowflake-side work.
- During: commit small; append every meaningful CoCo prompt and result to `docs/coco-playbook.md`.
- End of day: run `make test` and the phase check; tag; check spend (`cost-intelligence` skill); stop the Streamlit app and suspend search if idle.

Risks, spikes and open questions

Risk	Likelihood	Impact	Mitigation
Agent API not callable from Streamlit	Medium	High	Phase 0 spike on container runtime; fallback: run the agent via a Snowpark procedure the app calls
Cross-region inference disabled on the trial	Medium	High	Check and set in <code>00_account.sql</code> on day 0
AI_EXTRACT misreads generated PDFs	Medium	Medium	Clean typed layout; test on 5 PDFs before generating all 50; <code>MISSING_FIELDS</code> → QUERY path
Agent ignores JSON schema or skips tools	Medium	High	Strict instructions, validator + one retry, few-shot example in instructions, tool descriptions tuned via <code>agent-studio</code>
Rule values wrong (wording misread)	Low	High	Single primary wording; human verification gate; clause citation shown next to every amount
Credits run low before the finale (27–30 Oct)	Low	High	Resource monitor; efficient models; suspend idle services; keep ≥ \$100 in reserve
Masking bypassed through owner's-rights procedures	Medium	Medium	Tools return no PII or clinical text by design
Judges can't log in	Low	High	Dedicated judge user, tested from a private window; credentials in the submission form only
Scope creep	High	High	Stretch items only after all musts are green; cut line from the strategy doc

Open questions (answer before Phase 1)

- ☐ Solo or team? If team, who owns data/eval vs app vs agent?
- ☐ Which exact Arogya Sanjeevani wording (insurer + UIN) is the primary product?

- ☐ Does the submission portal ask for a demo video as well as the deck and repo? Check the dashboard.
- ☐ Which Snowflake region is the trial in (affects model availability)?